Lab 2: Handling Idempotency and Schema Changes with Auto Loader in Databricks

Lab Summary

In this lab, we test how **Databricks Auto Loader** processes new files incrementally and ensures **idempotency (no duplicate processing)**. We upload a second batch file into the **Auto Loader source volume** and rerun the streaming write code. The system processes **only the new file**, demonstrating **exactly-once processing**.

We also verify the data stored in the **Delta destination** and observe the `_rescued_data` **column**, which stores unexpected columns if schema changes occur.

Prerequisites

Before starting this lab, ensure the following are ready:

- Databricks workspace
- Unity Catalog enabled
- Volume created for Auto Loader
- Auto Loader streaming pipeline already configured
- Destination Delta location created
- Notebook containing Auto Loader streaming code

Steps to Complete Lab

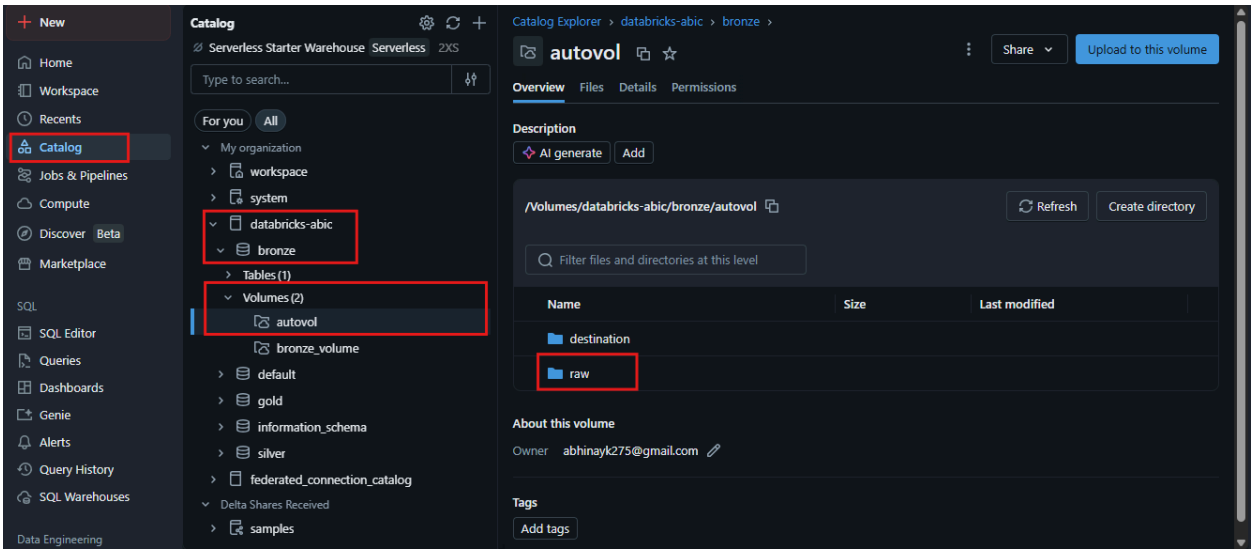
Task 1: Upload a New File to the Source Volume

Sub-task 1.1: Navigate to the Volume

1. Go to **Catalog** in Databricks.

2. Open the **Bronze catalog**.
3. Navigate to:

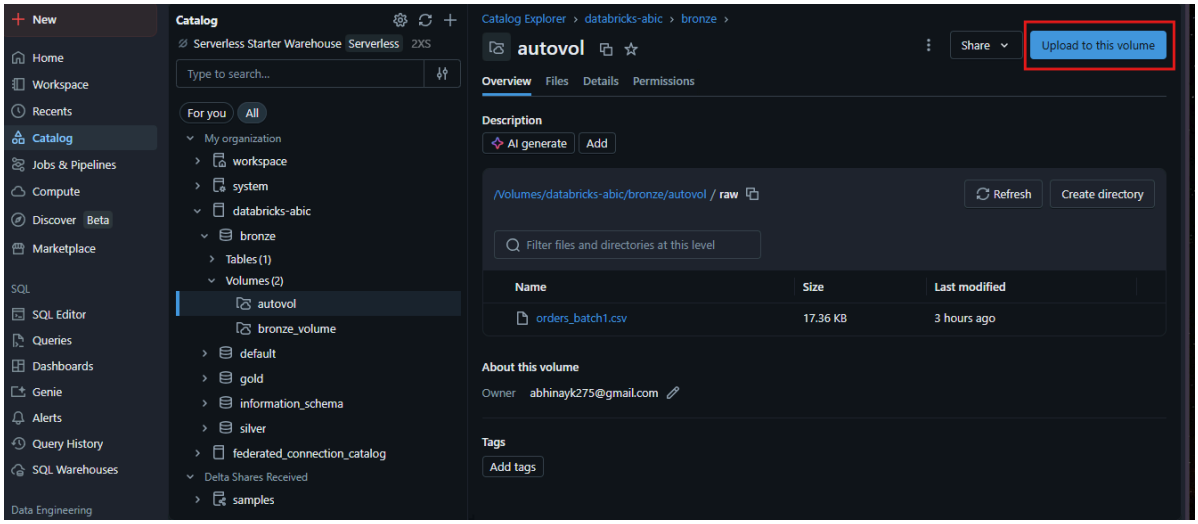
```
Volumes → auto_volume → raw
```



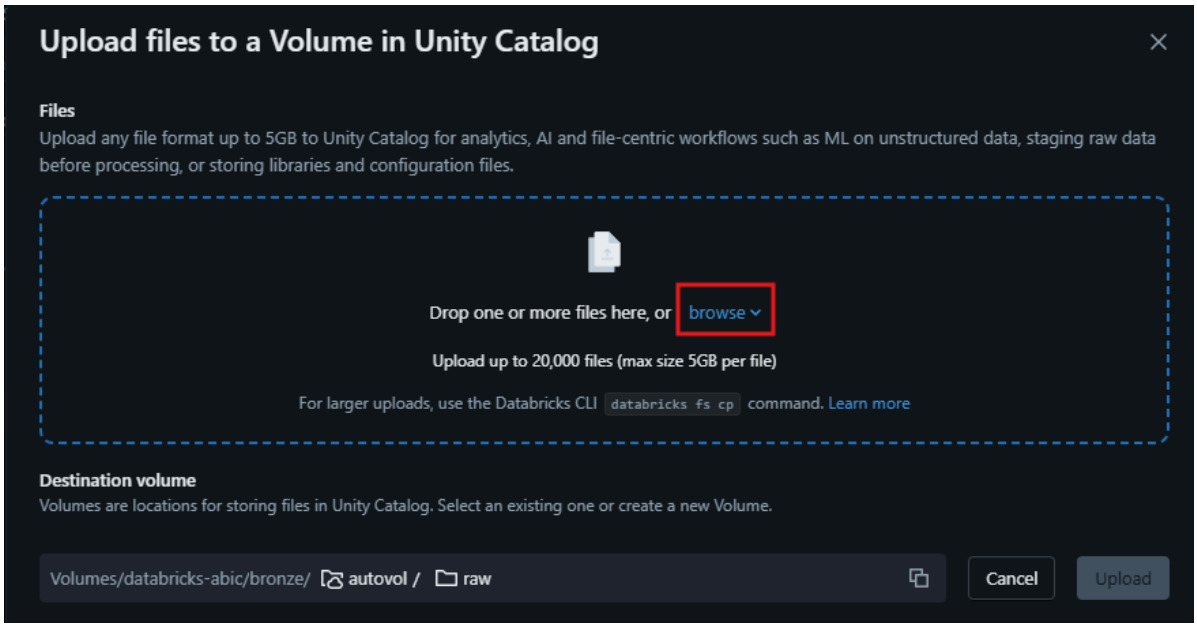
This is the **source location** from which Auto Loader reads files.

Sub-task 1.2: Upload the Second Batch File

1. Click **Upload**.



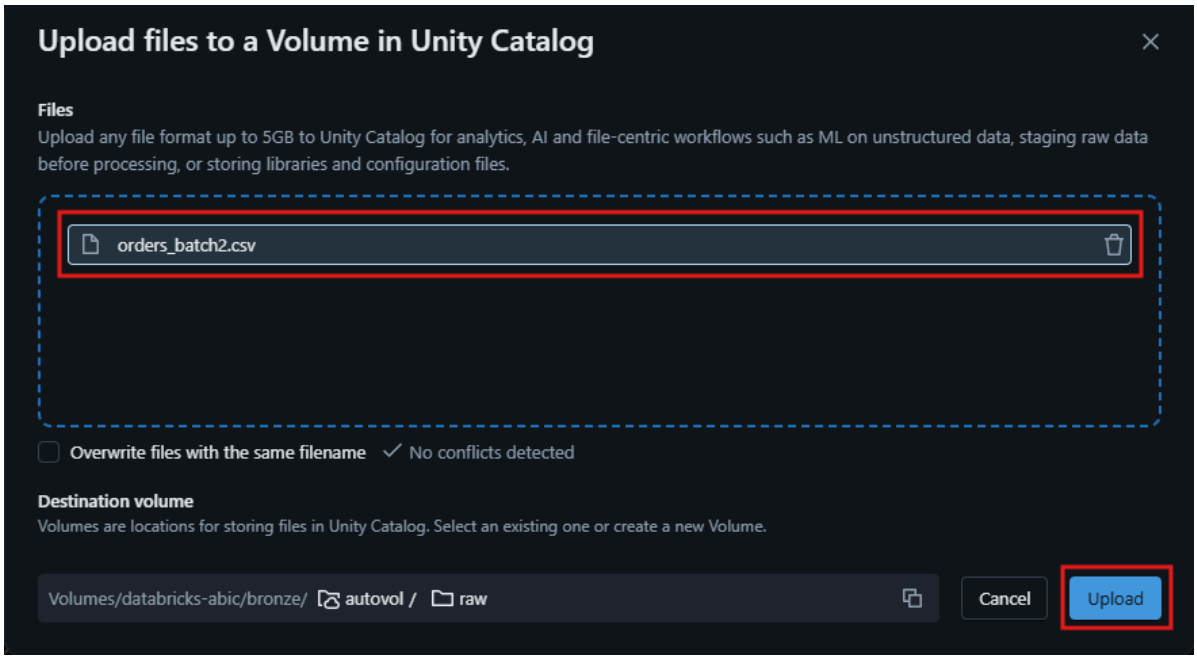
2. Select **Browse**.



3. Choose the file:

```
orders_batch2.json
```

4. Upload the file.



Important points:

- The file has the **same schema** as the first batch.
- Only **one new file** is uploaded.
- Auto Loader should process **only this file**.

Task 2: Rerun the Streaming Write Code

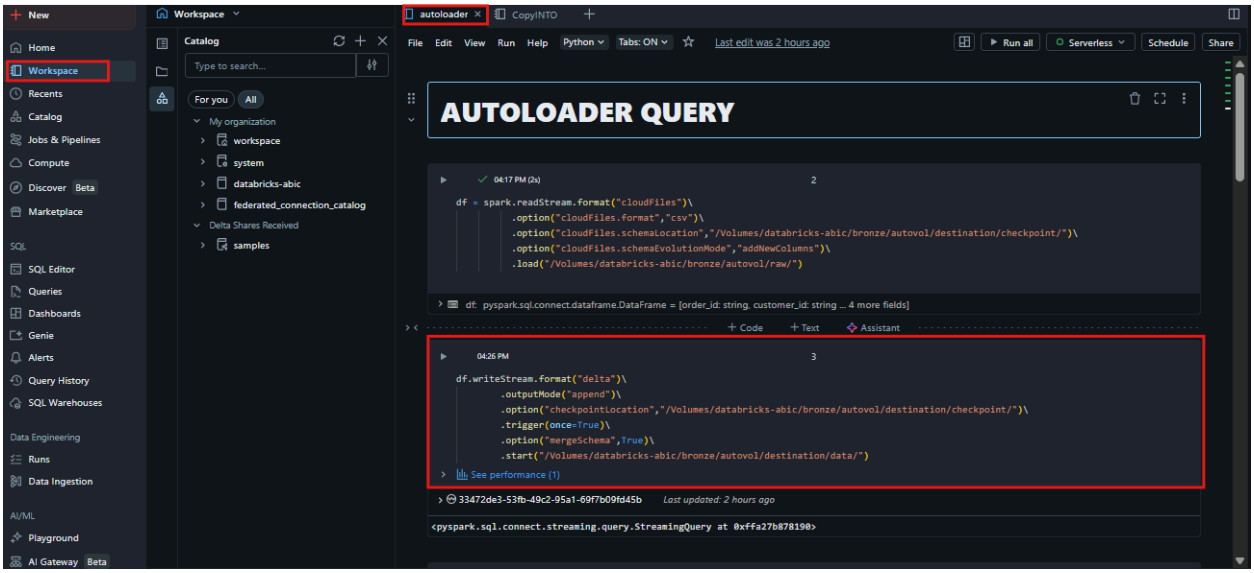
Sub-task 2.1: Open the Auto Loader Notebook

Navigate to the notebook where the **Auto Loader streaming pipeline** was created.

Important:

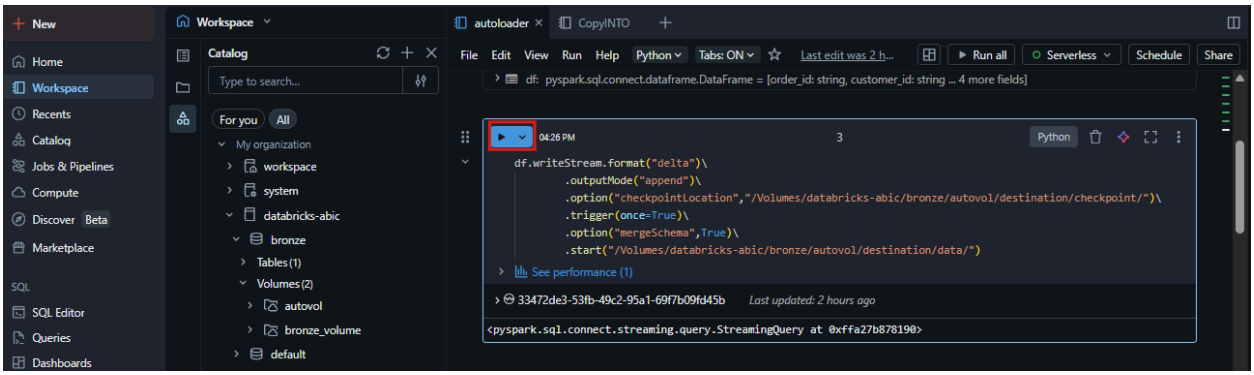
Do **NOT restart the query or reinitialize the stream**.

You only rerun the **write stream cell**.



Sub-task 2.2: Execute the Streaming Write Code

Run the code cell responsible for writing the stream.



Code:

```
df.writeStream.format("delta")\
        .outputMode("append")\
        .option("checkpointLocation","/Volumes/databricks-abi
c/bronze/autovol/destination/checkpoint/")\
        .trigger(once=True)\
        .option("mergeSchema",True)\
        .start("/Volumes/databricks-abic/bronze/autovol/desti
nation/data/")
```

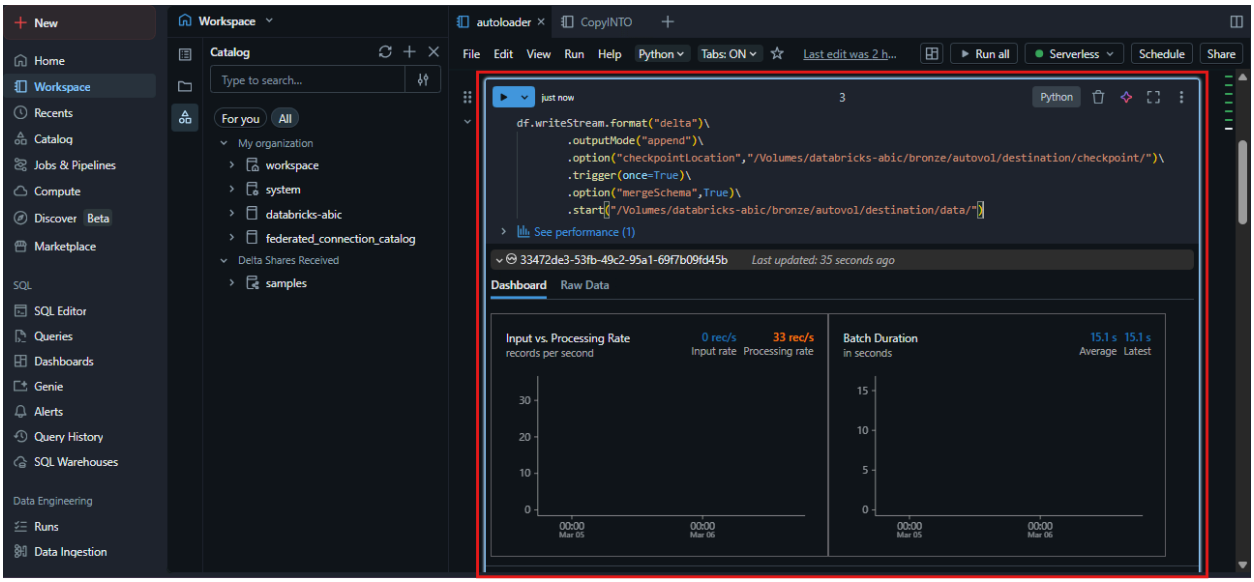
When the cell runs:

- The stream **initializes**
- Auto Loader checks the **checkpoint**
- It identifies **only new files**

Task 3: Observe Incremental Processing

After execution:

- The streaming graph will show **completed execution**.
- Only **one file** will be processed.



This happens because:

Idempotency

Auto Loader keeps track of processed files using **checkpoint metadata**.

Therefore:

- Previously processed files → **ignored**
- New files → **processed**

This guarantees **exactly-once processing**.

Task 4: Verify the Processed File in the Destination

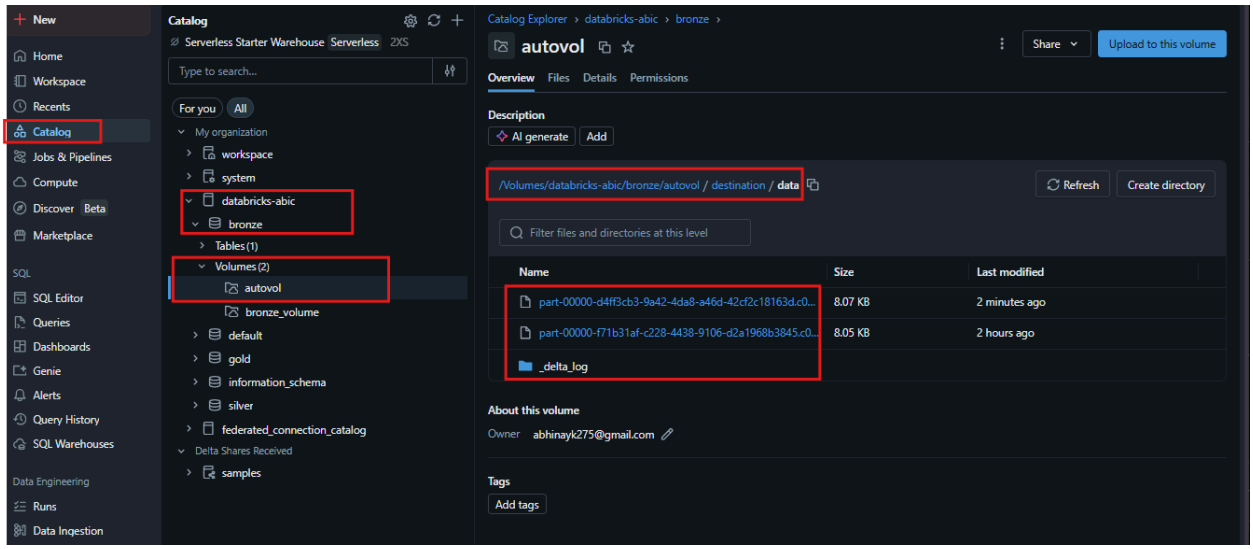
Sub-task 4.1: Navigate to Destination Volume

Go to:

```
Catalog → Bronze → Volumes → auto_volume
```

Open the **destination folder**.

You should now see **one additional file** generated from the newly processed batch.



This confirms:

- Only the **new batch file** was processed
- Previous files were **not reprocessed**

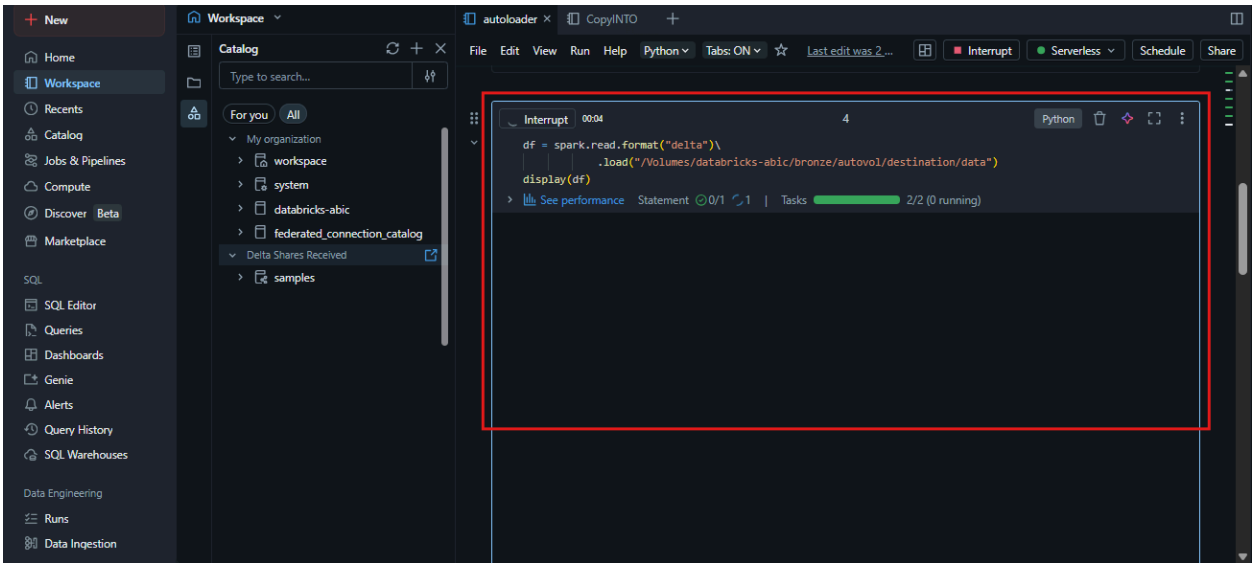
Task 5: Read the Destination Data as a Static DataFrame

Now we verify the stored data.

Sub-task 5.1: Read the Delta Data

Run the following code in a notebook cell:

```
df = spark.read.format("delta")\
            .load("/Volumes/databricks-abic/bronze/autovol/de
stination/data")
display(df)
```



Important:

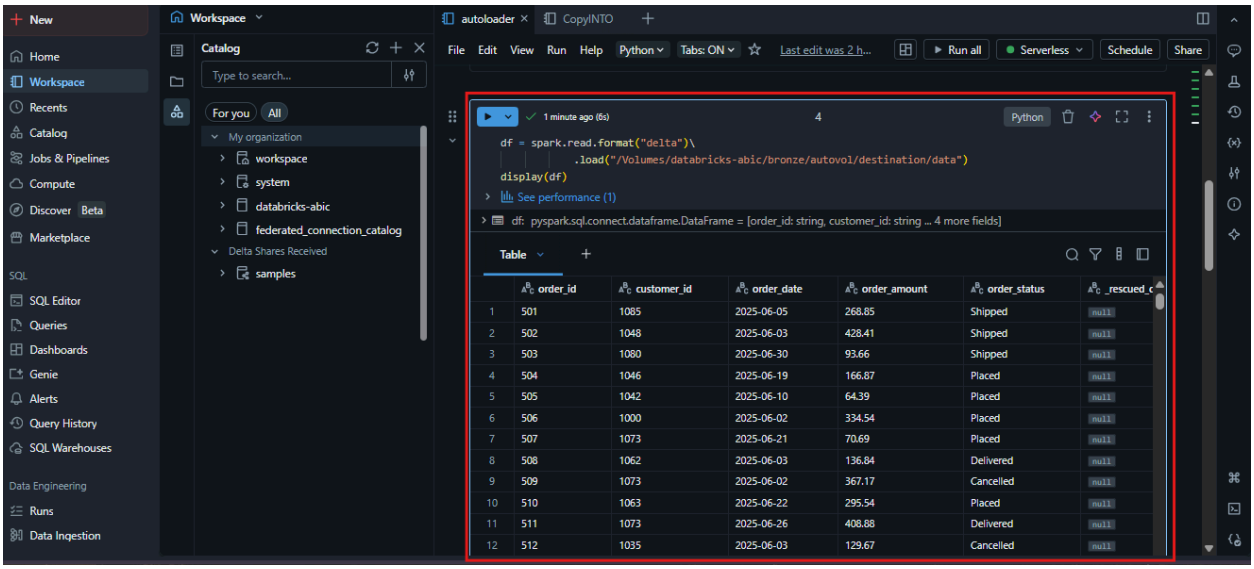
- This is a **static read**
- We are **not reading streaming data**

Sub-task 5.2: Inspect the Output

You should see:

- Clean structured data
- Records from both batch files
- An additional column called:

```
_rescued_data
```



Task 6: Understand the `_rescued_data` Column

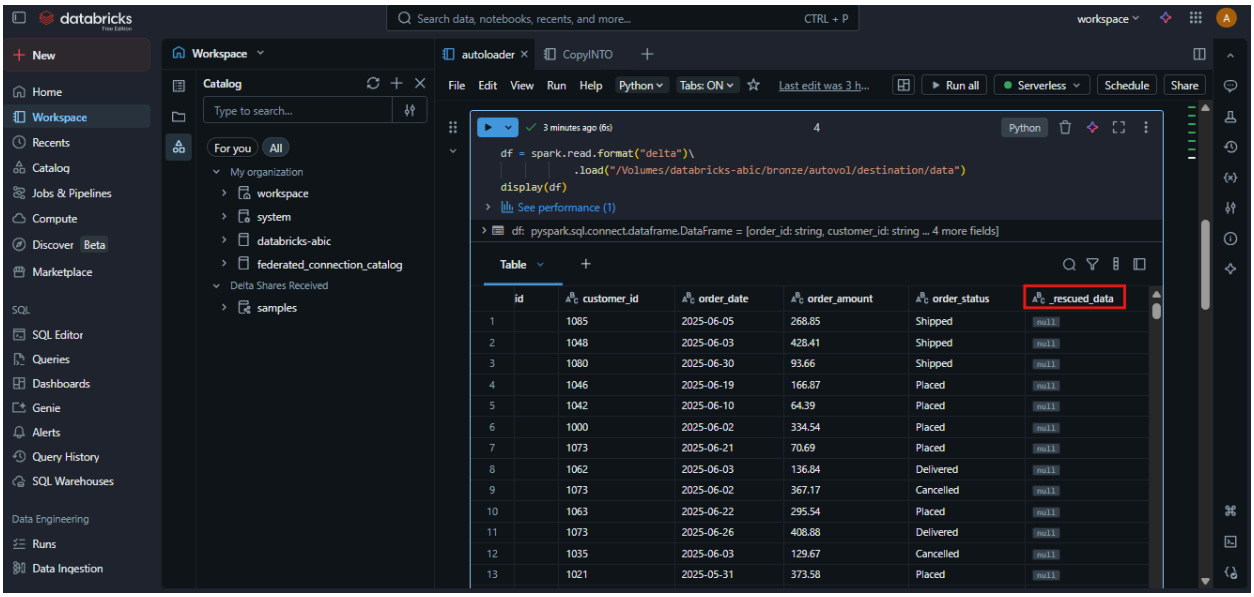
The `_rescued_data` column is automatically created by **Auto Loader**.

Purpose:

- Stores **unexpected columns**
- Handles **schema evolution**

Current status:

```
_rescued_data = null
```



Why?

Because:

- Both files follow the **same schema**
- No new columns were introduced

Therefore:

```
Nothing needs to be rescued
```

Outcome

After completing this lab:

You successfully learned:

- How **Auto Loader processes only new files**
- The concept of **Idempotency**
- **Exactly-once processing** in streaming pipelines
- How to verify processed data using **Delta tables**
- The role of `_rescued_data` **in schema evolution**